



Machine Learning in Production

Testing in Production

Machine Learning in Production • Christian Kaestner & Bogdan Vasilescu, Carnegie Mellon University • Fall 2025



Changelog 
@changelog · [Follow](#)



“Don’t worry, our users will notify us if there’s a problem”



10:03 AM · Jun 8, 2019



1.9K



Reply



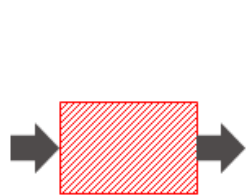
Copy link

[Read 11 replies](#)

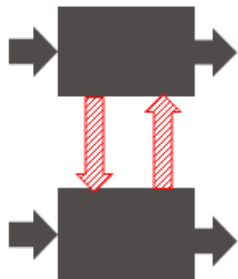
From Unit Tests to Testing in Production

Unit Test, Integration Tests, System Tests

These are all testing *before* deployment --- Manual or automated



Unit testing



Integration testing



System testing



Acceptance
testing
(Demonstration)

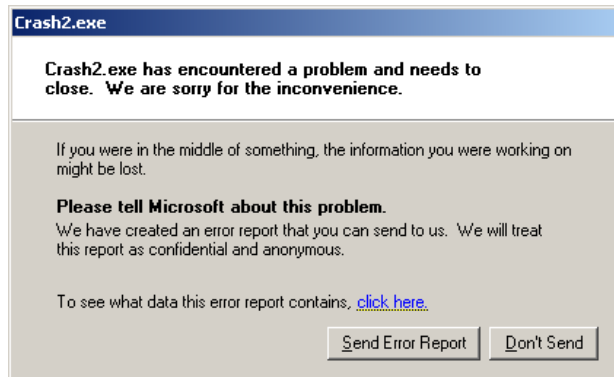
Beta Testing: Move towards testing during deployment

Early release to select users, asking them to send feedback or report issues.



Crash Telemetry: "Bugs" in Production

- **Telemetry:** The automated collection, transmission, and analysis of data about the performance, behavior, and usage of an application or system in real time.
- With internet availability, send crash reports home to identify problems "in production".



Model Assessment in Production

Recap: Limitations of Offline Model Eval

Training and test data drawn from the same population

- **i.i.d.: independent and identically distributed**
- leakage and overfitting problems quite common

Is the population representative of production data?

- If not or only partially or not anymore: Does the model generalize beyond training data?

Model Assessment in Production

Live observation in the running system

Need **telemetry** to evaluate quality – automated collection and monitoring of data on model performance, data quality, and user behavior to ensure continuous model accuracy, detect drift, and maintain system health in real-time.

Discuss how to collect data

How do you gather data **without being intrusive** (i.e., labeling outcomes), **without harming user** experience?

- Was the house price predicted correctly?
- Was the ranking of search results good?
- Did the profanity filter remove the right blog comments?
- Was there cancer in the image?
- Was a Spotify playlist good?
- Was the weather prediction good?
- Was the translation correct?
- Did the self-driving car break at the right moment? Did it detect the pedestrians?

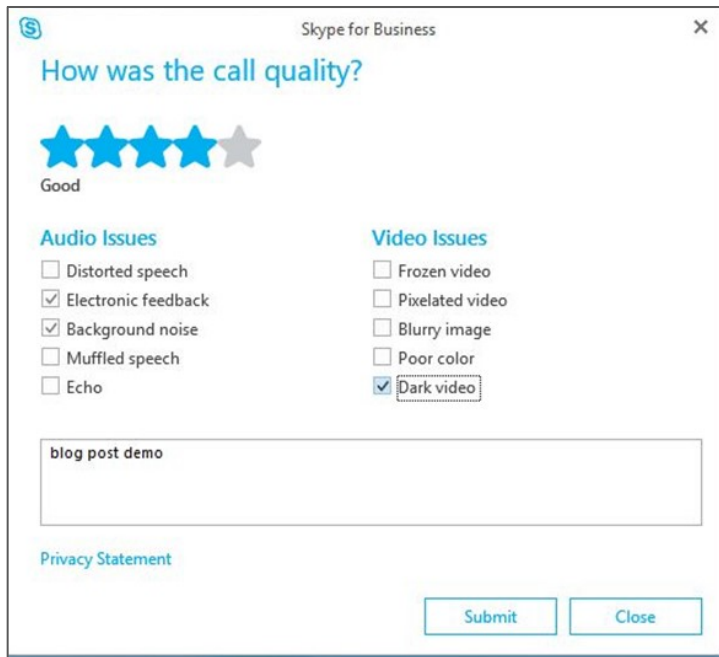


Telemetry strategy: Manually Label Production Samples

Similar to labeling learning and testing data, have human annotators



Telemetry strategy: Explicit Report



Skype for Business

How was the call quality?

★★★★☆
Good

Audio Issues

- ☐ Distorted speech
- ☒ Electronic feedback
- ☒ Background noise
- ☐ Muffled speech
- ☐ Echo

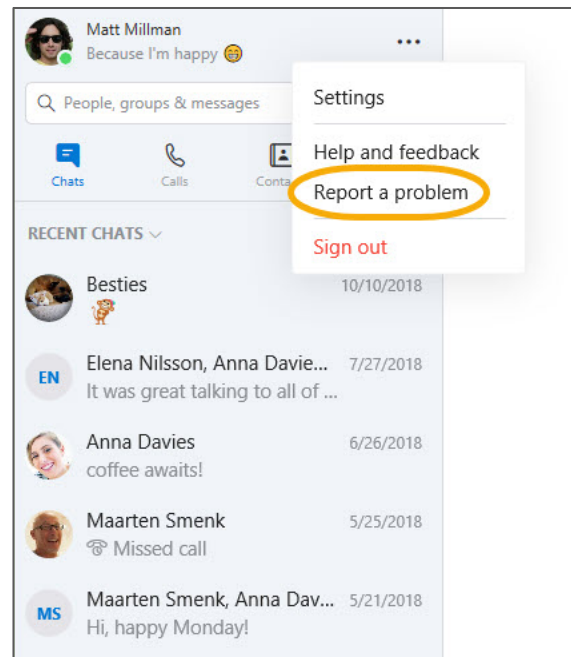
Video Issues

- ☐ Frozen video
- ☐ Pixelated video
- ☐ Blurry image
- ☐ Poor color
- ☒ Dark video

blog post demo

[Privacy Statement](#)

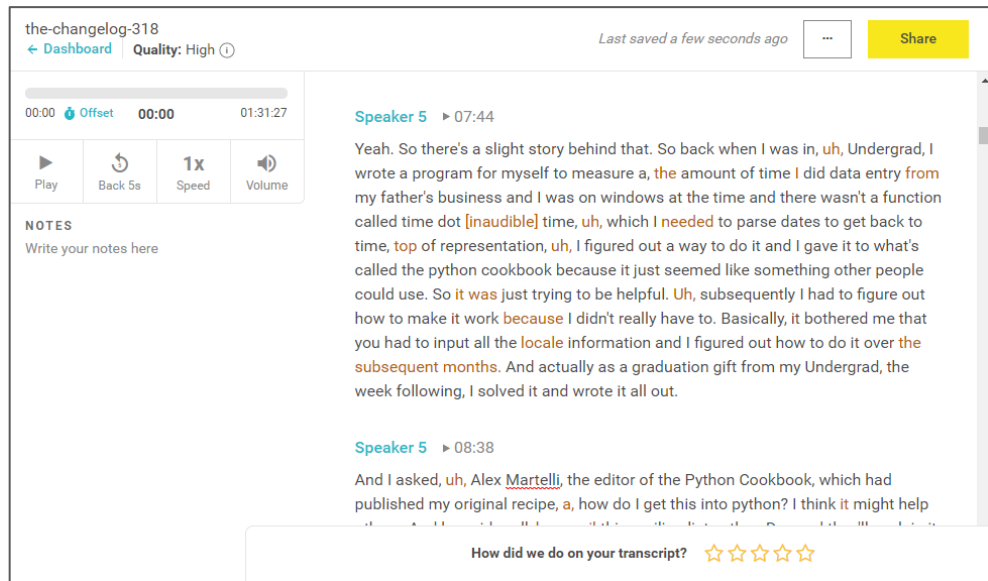
[Submit](#) [Close](#)



Expect only sparse feedback and expect negative feedback over-proportionally

Telemetry strategy: Implicit Feedback

Collect user editing ("human-AI collaboration"); Can have indicators for low confidence predictions



Telemetry strategy: Wait-and-See (Retrospective)

DFW ↔ SFO
1659 of 1687 flights

Nov 16
Wednesday

Advice: **Watch** [Learn more](#) ⓘ

Create a price alert

Stops

- ☒ nonstop
- ☒ 1 stop
- ☒ 2+ stops

Times

Take-off Dallas

Prices may fall within 7 days – Watch

Our model strongly indicates that fares will fall during the next 7 days. This forecast is based on analysis of historical price changes and is not a guarantee of future results.

Create a price alert

Summary: Telemetry Strategies

Ordered by how much efforts needed:

- Manual/crowdsourced labeling on production samples
- Explicit report: Ask users or allow users to complain
- Observe user reaction (implicit feedback)
- Wait and see

Measuring Model Quality with Telemetry

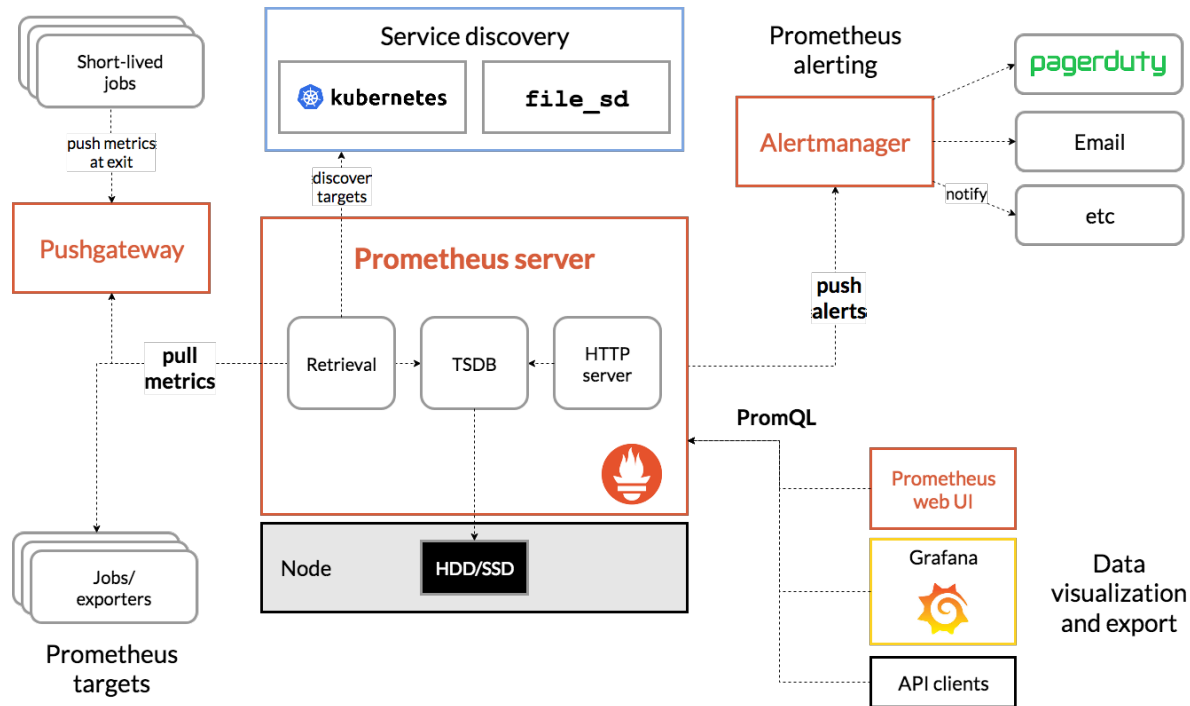
Three steps, again on measurements: (1) Metric, (2) Data collection (telemetry), (3) Operationalization

- Telemetry can provide insights for correctness
 - sometimes very accurate labels for real unseen data
 - sometimes only mistakes
 - sometimes delayed
 - often just samples
 - often just weak proxies for correctness
- Often sufficient to *approximate* precision/recall or other model-quality measures
- Mismatch to (static) eval set may indicate stale or unrepresentative data
- Trend analysis can provide insights even for inaccurate proxy measures

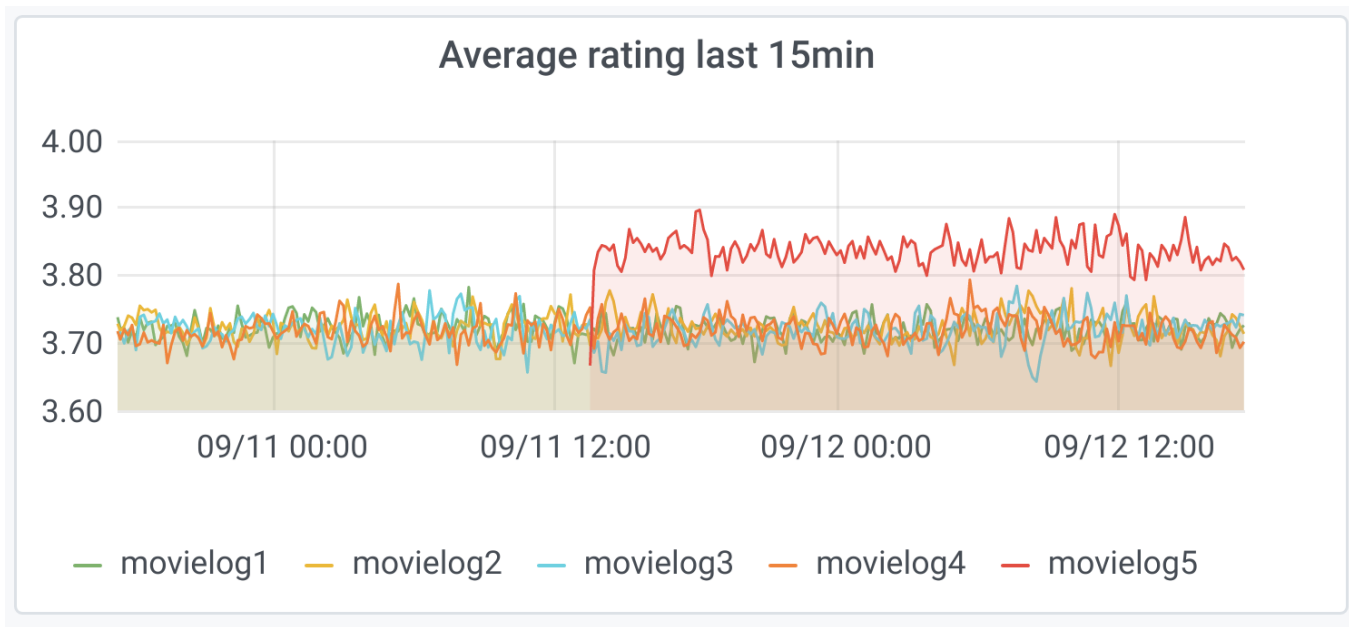
Monitoring Model Quality in Production

- Monitor model quality together with other quality attributes (e.g., uptime, response time, load)
- Set up automatic alerts ...
 - When model quality drops
 - Watch for jumps after releases (roll back after negative jump)
 - Watch for slow degradation (Stale models, data drift, feedback loops, adversaries)
- Debug common or important problems
 - Monitor characteristics of requests
 - Mistakes uniform across populations?
 - Challenging problems → refine training, add regression tests

Monitoring services: Prometheus and Grafana



Grafana screenshot from Movie Recommendation Service



Many different monitoring possibilities, e.g. Detecting Drift

Deploy monitoring at ...

Training data

- Schema & distribution of incoming data
- Distribution of labels

Requests & predictions

- Schema & distribution of requests
- Distribution of predictions
- Quality of predictions

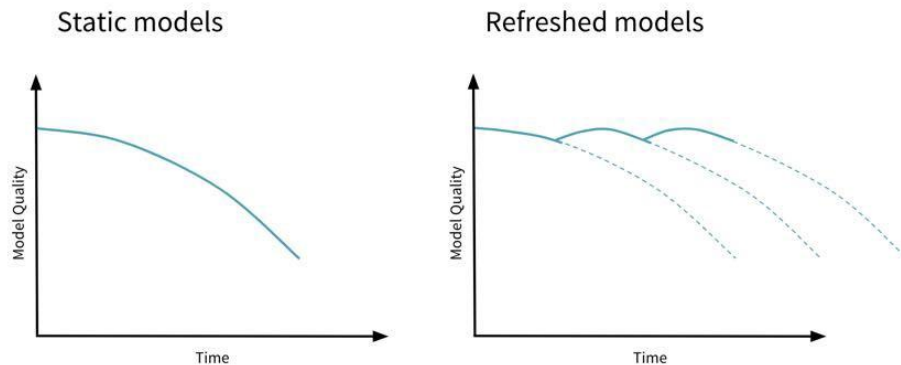
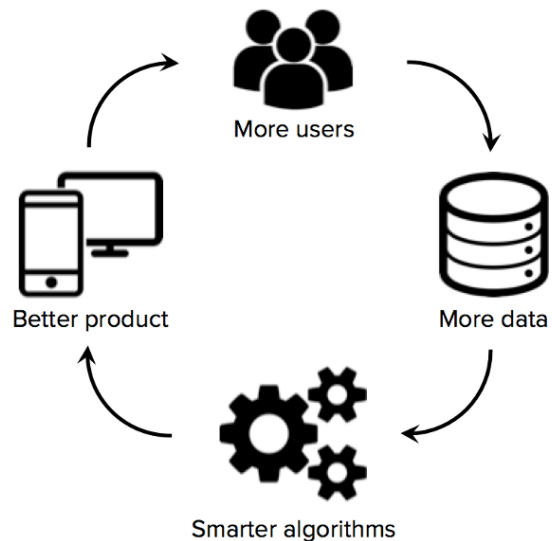


Image source: Joel Thomas and Clemens Mewald. [Productionizing Machine Learning: From Deployment to Drift Detection](#). Databricks Blog, 2019

Monitoring without Ground Truth: Invariants/Assertions to Assure with Telemetry

- Consistency between multiple sources
 - e.g., multiple models agree, multiple sensors agree
 - e.g., text and image agree
- Physical domain knowledge
 - e.g., cars in video shall not flicker,
 - e.g., earthquakes should appear in sensors grouped by geography
- Domain knowledge about unlikely events
 - e.g., unlikely to have 3 cars in same location
- Stability
 - e.g., object detection should not change with video noise
- Input conforms to schema (e.g. boolean features)
- And all invariants from model quality lecture, including capabilities

Telemetry for Training: The ML Flywheel



graphic by [CBInsights](#)

Engineering Challenges for Telemetry

Data volume and operating cost

- e.g., record "all AR live translations"?
- reduce data through sampling
- reduce data through summarization (e.g., extracted features rather than raw data; extraction client vs server side)

Isolating feedback for specific ML component + version

Biased sampling / Rare events

Privacy (related: Offline deployments)



Revisiting Model Quality vs System Goals

Model Quality vs System Goals

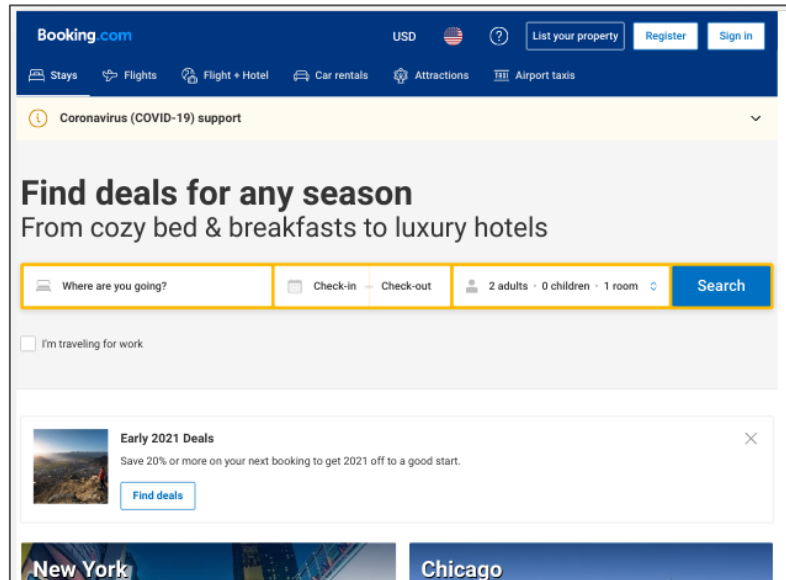
Telemetry can approximate model accuracy

Telemetry can directly measure system qualities, leading indicators, user outcomes

- define measures for "key performance indicators"
- clicks, buys, signups, engagement time, ratings
- operationalize with telemetry

Model Quality vs System Quality: booking.com

What's the system goal?

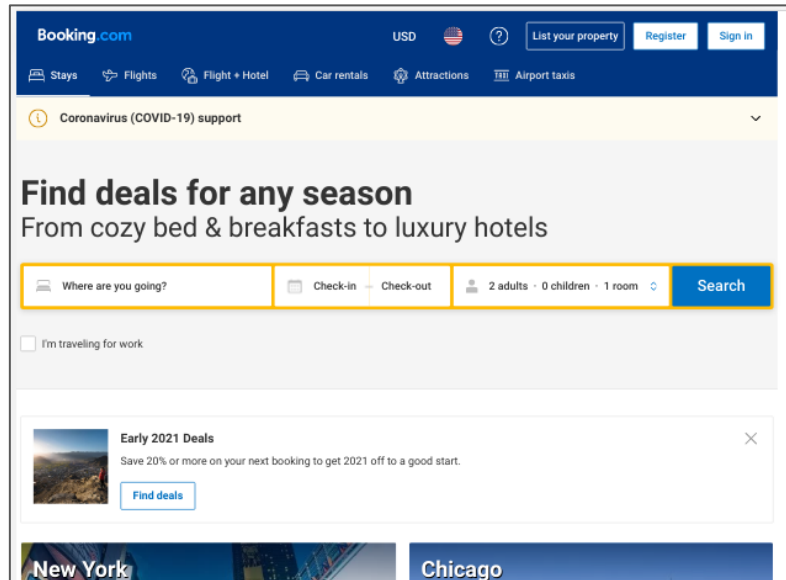


The screenshot displays the Booking.com homepage. At the top, there's a dark blue navigation bar with the Booking.com logo, currency (USD), a help icon, and links for 'List your property', 'Register', and 'Sign in'. Below this is a horizontal menu with icons for 'Stays', 'Flights', 'Flight + Hotel', 'Car rentals', 'Attractions', and 'Airport taxis'. A yellow banner for 'Coronavirus (COVID-19) support' is visible. The main section features the heading 'Find deals for any season' with the subtext 'From cozy bed & breakfasts to luxury hotels'. Below this is a search bar with fields for 'Where are you going?', 'Check-in', 'Check-out', and a dropdown for '2 adults · 0 children · 1 room', followed by a blue 'Search' button. A checkbox for 'I'm traveling for work' is also present. At the bottom, there are two promotional banners: 'Early 2021 Deals' with a 'Find deals' button, and a banner for 'New York' and 'Chicago'.

Bernardi, Lucas, et al. "150 successful machine learning models: 6 lessons learned at Booking.com." In Proc. Int'l Conf. Knowledge Discovery & Data Mining, 2019.

Model Quality vs System Quality: booking.com

Which telemetry strategies would you use?



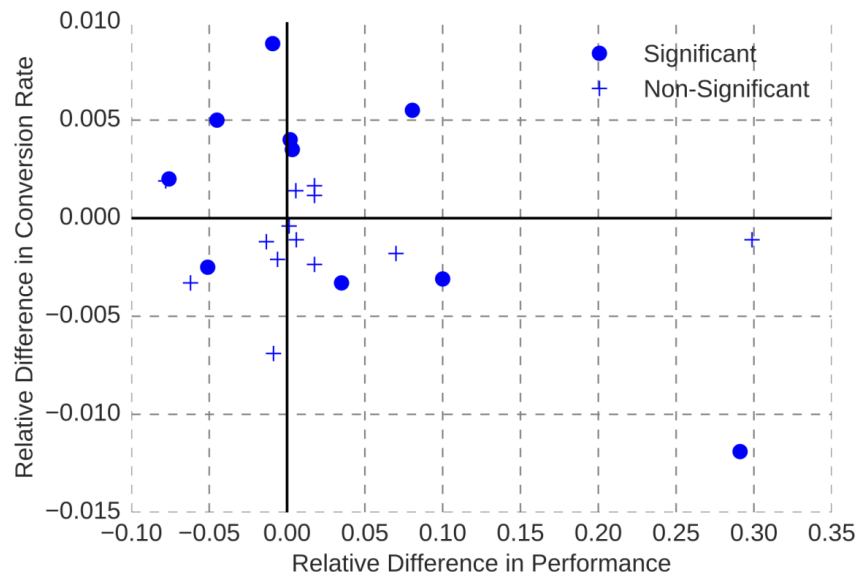
Bernardi, Lucas, et al. "150 successful machine learning models: 6 lessons learned at Booking.com." In Proc. Int'l Conf. Knowledge Discovery & Data Mining, 2019.

Possible causes of model vs system conflict?

Observed no correlation between offline performance gain vs. business value gain, across many different models with different goals.

Possible reasons:

- Value Performance Saturation
- Uncanny Valley effect (*e.g.*, "creepy AIs")
- Proxy Over-optimization (*e.g.*, *click-through rate approximating conversion*)



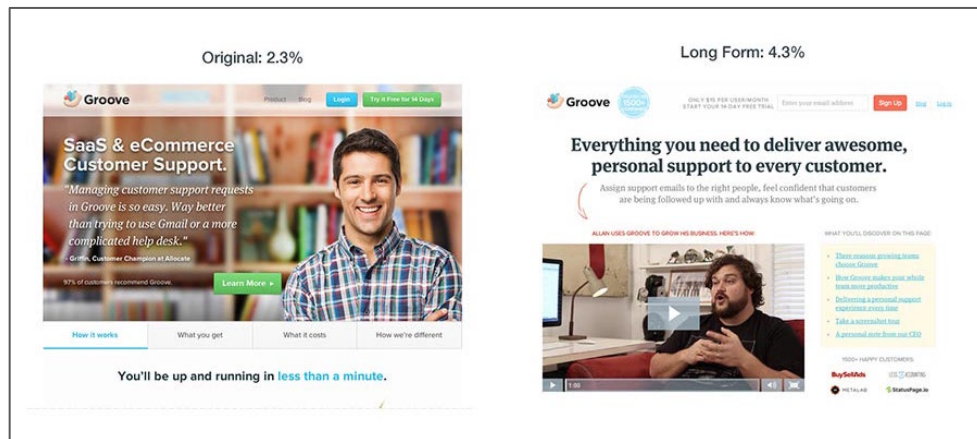
Bernardi, Lucas, et al. "150 successful machine learning models: 6 lessons learned at Booking.com." In Proc. Int'l Conf. Knowledge Discovery & Data Mining, 2019.

Experimenting in Production

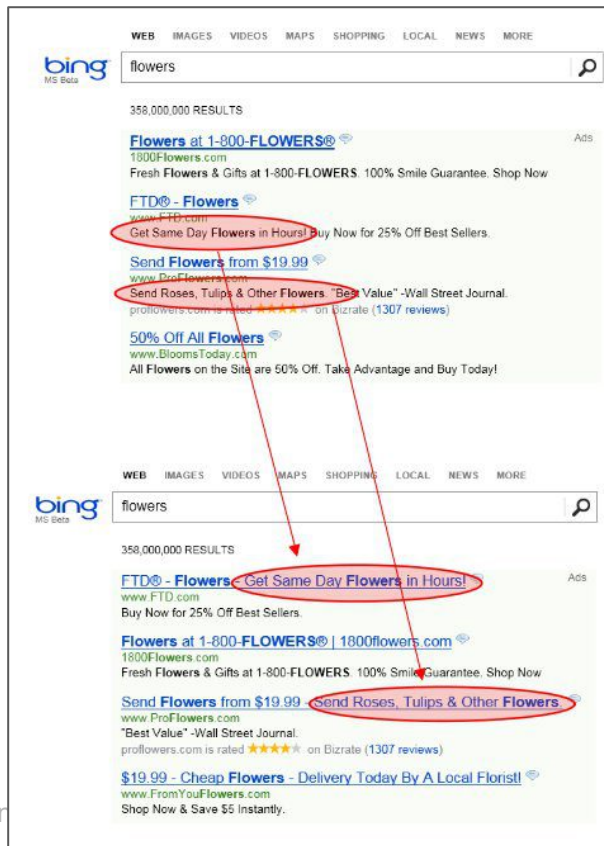
- A/B experiments
- Shadow releases / traffic teeing
- Blue/green deployment
- Canary releases
- Chaos experiments

A/B Experiments: What if...?

- ... we hand plenty of subjects for experiments
- ... we could randomly assign to treatment/ control group without them knowing
- ... we could analyze small individual changes and keep everything else constant



Bing Experiment



- Experiment: Ad display at Bing
- Suggestion deprioritized and left in the backlog for over 6 months
- Implemented & A/B tested in production
- Within 2h *revenue-too-high* alarm triggered.
 - Typically suggest serious bug (e.g., double billing)
 - But valid here: Revenue increase by 12% - \$100M annually in US
 - Did not hurt user-experience metrics
- Simple adjustment became one of the most profitable ideas in Bing's history!

From: Kohavi, Ron, Diane Tang, and Ya Xu. "[Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing](#)." 2020.

Implementing A/B Testing

Implement alternative versions of the system

- using feature flags (decisions in implementation)
- separate deployments (decision in router/load balancer)

Map users to treatment group

- Randomly from distribution
- Static user - group mapping
- Online service (e.g., [launchdarkly](#), [split](#))

Monitor outcomes *per group*

- Telemetry, sales, time on site, server load, crash rate

Feature Flags (Boolean flags)

```
if (features.enabled(userId, "one_click_checkout")) {  
    // new one click checkout function  
} else {  
    // old checkout functionality  
}
```

- Good practices: tracked explicitly, documented, keep them localized and independent
- External mapping of flags to customers, who should see what configuration
 - e.g., 1% of users sees `one_click_checkout`, but always the same users; or 50% of beta-users and 90% of developers and 0.1% of all users

```
def isEnabled(user): Boolean = (hash(user.id) % 100) < 10
```

Confidence in A/B Experiments (statistical tests)

Group A

*classic personalized content
recommendation model*

2158 Users

average 3:13 min time on site

Group B

*updated personalized content
recommendation model*

10 Users

average 3:24 min time on site

What's the problem of comparing the average?

Concurrent A/B testing

Multiple experiments at the same time

- Independent experiments on different populations – interactions not explored
- Multi-factorial designs, well understood but typically too complex, e.g., not all combinations valid or interesting
- Grouping in sets of experiments (layers)

Other Experiments in Production

Chaos experiments

Shadow releases / traffic teeing

Canary releases

Chaos Experiments

Deliberate introduction of faults in production to test **robustness**.



Shadow releases / traffic teeing

Run both models in parallel

Use predictions of old model in production

Compare differences between model predictions

If possible, compare against ground truth labels/telemetry

Examples?

Canary Releases

Release new version to small percentage of population (like A/B testing)

Automatically roll back if quality measures degrade

Automatically and incrementally increase deployment to 100% otherwise



Advice for Experimenting in Production

Minimize *blast radius* (canary, A/B, chaos expr)

Automate experiments and deployments

Allow for quick rollback of poor models (continuous delivery, containers, load balancers, versioning)

Make decisions with confidence, compare distributions

Monitor, monitor, monitor

Summary

Production data is ultimate unseen validation data

Both for model quality and system quality

Telemetry is key and challenging (design problem and opportunity)

Monitoring and dashboards

Many forms of experimentation and release (A/B testing, shadow releases, canary releases, chaos experiments, ...) to minimize "blast radius"; gain confidence in results with statistical tests

Further Readings

- On canary releases: Alec Warner and Štěpán Davidovič. “[Canary Releases](#).” in [The Site Reliability Workbook](#), O’Reilly 2018
- Everything on A/B testing: Kohavi, Ron. [Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing](#). Cambridge University Press, 2020.
- A/B testing critiques: Josh Constine. [The Morality Of A/B Testing](#). Blog 2014; the [Center of Humane Technology](#); and the Netflix documentary [The Social Dilemma](#)
- Ori Cohen “[Monitor! Stop Being A Blind Data-Scientist](#).” Blog 2019
- Jens Meinicke, Chu-Pan Wong, Bogdan Vasilescu, and Christian Kästner. [Exploring Differences and Commonalities between Feature Flags and Configuration Options](#). In Proceedings of the Proc. International Conference on Software Engineering ICSE-SEIP, pages 233–242, May 2020.